

Threading & Concurrency (.NET / OS Concepts)

DAY - 1

1	Foundations	Process vs thread vs AppDomain; Why threading (responsiveness, parallelism, throughput); Thread lifecycle and states; UI/main thread concept; Foreground vs background threads; Context-switching cost; CPU cores vs threads	Conceptual session. No heavy code. Use diagrams.
---	-------------	--	--

1. Process

Definition: A Process is a running instance of a program with its own memory space, resources and execution context.

Program -> OS loads executable -> Process created -> Main Thread created -> Execution begins.

Memory Layout:

- Code/Text Segment: instructions, functions
- Data Segment: global/static variables
- Heap: dynamic memory
- Stack: local variables, function frames

Key points:

- Every process has at least one thread.
- Processes have isolated memory spaces.
- IPC examples: pipes, sockets, shared memory.

2. Thread

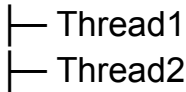
Definition: Smallest unit of CPU execution inside a process.

Shared Resources: Code, Heap, Data Segment, Files, Address Space.

Private Resources: Stack, Registers, Program Counter.

Hierarchy:

Process



Thread advantages:

- Responsiveness
- Concurrency
- Better CPU utilization
- Parallel execution on multi-core CPUs

3. AppDomain (.NET)

Logical isolation boundary inside a .NET process (CLR construct).

Hierarchy:

OS -> Process -> AppDomain -> Threads

Default: Every .NET process starts with one Default AppDomain.

Useful APIs:

`AppDomain.CurrentDomain.FriendlyName`

`AppDomain.CreateDomain()`

`CreateInstanceAndUnwrap()`

`MarshalByRefObject`: enables cross-AppDomain communication by reference (proxy).

Modern Note: .NET Core / .NET 5+ mainly uses `AssemblyLoadContext`.

4. Why Threading?

Responsiveness: Keep application interactive during long work.

Parallelism: True simultaneous computation using multiple CPU cores.

Concurrency: Manage multiple tasks during same time period.

Throughput: Amount of work completed per unit time.

Common APIs:

Responsiveness -> `Task.Run`, `async/await`, `Thread`

Parallelism -> `Parallel.For`, `Parallel.ForEach`, `AsParallel`

Concurrency -> `Task.WhenAll`, `Thread`, `async/await`

Throughput -> `ThreadPool`, batching, `async I/O`

5. Thread Lifecycle & States

States: (`thread.ThreadState`)

- Unstarted: Created but `Start()` not called
- Runnable/Ready: Waiting for scheduler
- Running: Currently executing
- WaitSleepJoin: Sleep/Join/Wait state
- Stopped: Finished execution

Lifecycle:

Unstarted -> `Start()`

Ready -> Running

Running -> WaitSleepJoin

WaitSleepJoin -> Ready

Running -> Stopped

`Join()`: Blocks the calling thread until target thread completes.

6. UI/Main Thread

UI Thread handles:

- Button clicks
- Mouse/keyboard events
- Rendering/redrawing

Blocking UI thread causes freezes.

Fix: Use `Task.Run` / `async-await` to move heavy work off UI thread.

7. Foreground vs Background Threads

Foreground Thread:

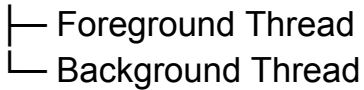
- Keeps process alive
- Important work

Background Thread:

- Does not keep process alive
- Killed when all foreground threads finish

Important:

Thread



Background thread != ThreadPool thread

Default new Thread() => Foreground

8. ThreadPool

Managed pool of reusable worker threads maintained by CLR.

Purpose:

- Avoid expensive thread creation/destruction
- Improve throughput

Used by:

Task.Run

Parallel.For

async/await (often indirectly)

9. Context Switching

Definition: Saving one execution state and loading another.

Context contains:

- Registers
- Stack Pointer
- Program Counter
- Execution State

Thread switch: cheaper (shared memory)

Process switch: costlier (separate address spaces, page tables)

Too many threads increase switching overhead.

10. CPU Cores vs Threads

CPU Core: Physical execution unit.

CPU Thread / Logical Processor: Logical execution context exposed by a core (Hyper-Threading/SMT).

Important Distinction:

1. Software Thread (.NET new Thread())
2. OS Thread (scheduler-managed)
3. Hardware Thread / Logical Processor (CPU capability)

Execution Flow:

Your App

- > Software Threads
- > OS Scheduler
- > CPU Logical Processors
- > Physical CPU Cores

Example:

8 cores, 16 logical processors.

100 software threads != 100 simultaneous executions.

11. Quick Notes

- Process always has at least one thread.
- Threads share process memory; processes do not.
- Concurrency != Parallelism.
- Parallelism needs multiple cores.
- Background threads do not keep process alive.
- ThreadPool improves efficiency through reuse.
- Process context switch > Thread context switch cost.

DAY - 2

2	Low-level Thread API	<code>System.Threading.Thread</code> (Start, Join, Sleep); Passing parameters; <code>ParameterizedThreadStart</code> ; Why <code>Thread.Abort</code> is deprecated; <code>ThreadPool.QueueUserWorkItem</code> ; When (not) to use <code>Thread</code> directly	Teach as historical baseline. Most modern code uses <code>Task</code> .
---	----------------------	--	---

System.Threading.Thread is low-level class used to create and manage an operating system thread.

1. **Thread.Start()** - **Purpose:** Instructs the operating system to begin executing the thread.

new Thread() — Unstarted state — Memory is allocated — thread.start() — Runnable state waits in threadpool until the OS thread scheduler allocate CPU time

2. **Thread.Sleep** - **Purpose:** Pauses the current executing thread for a specified period.
 - **Thread.Sleep(1000) (Actual Sleep)**
 - **Action:** The thread is completely removed from the CPU execution queue.
 - **State Change:** Transitions from Running to the **WaitSleepJoin** state.
 - **OS View:** The OS scheduler marks this thread as *unavailable* for the duration. It will not run until the timer expires.
 - **Thread.Sleep(0) (The Polite Yield)**
 - **Action:** The thread yields its remaining time slice, saying, "If there is another thread of equal priority waiting, it can skip ahead of me right now. If not, I'll keep running."
 - **State Change:** Stays in the **Runnable (Ready)** state. It never drops into a blocked state.

- **OS View:** If no other threads are waiting, execution continues immediately. If one is waiting, the OS swaps your thread out and puts it at the end of the line.
- **Use Case:** Used in intensive background computing loops (like **SpinLocks**) to ensure the thread doesn't hog the entire CPU core.
- **Modern .NET Tip:** Use **Thread.Yield()**, which is preferred as it allows *any* ready thread to run on that CPU core, even if it has a lower priority.

Thread doing its work — identifies Thread.Sleep() — that particular thread goes to sleep — state WaitSleepJoin — yield its current CPU time slice to other thread in queue

3. Thread.Join() - Purpose: Blocks the calling thread until the target thread finishes execution. Join() is used for synchronization

If Thread A calls ThreadB.Join(), Thread A will stop completely and wait at that line of code until Thread B completes its work and enters the Stopped state.

```
Static void Main(){ //main thread created
    Thread t1 = new Thread(); //main thread is t1 thread
    Thread t2 = new Thread(); //another new thread t2
    Thread t3 = new Thread(); //another new thread t3
    t1.start();
    t2.start();
    t3.start();
    t2.join(); //blocked t1 (main thread) t2 will does it work and unblock t1
    t3.join(); //blocked t1 (main thread) t3 will does it work and unblock t1
}
```

Why use it?

It ensures that dependent tasks happen in the correct sequence. For example, you cannot analyze data (Thread A) until the data-loading thread (Thread B) is completely finished.

Passing parameters to threads

1. No parameter thread
2. ParameterizedThreadStart (only one parameter)
3. Lambda Expression (multiple parameters)
4. Custom object /DTO (complex data transfer, large input payload)
5. Modern .NET approach (using Task.Run(), accept multiple parameter)

No Parameter

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(Work);
    }

    static void Work()
    {
        Console.WriteLine("No Parameter");
    }
}
```

ParameterizedThreadStart

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(new ParameterizedThreadStart(Work));
        t1.Start("Hello");
    }

    static void Work(object data)
    {
        Console.WriteLine(data);
    }
}
```


Lambda Expression

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(()=>
        {
            Work(1, "Samyuktha");
        });
        t1.Start();
    }
    static void Work(int id, string name)
    {
        Console.WriteLine($"{id} - {name}");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        int x = 10;
        Thread t1 = new Thread(() =>
        {
            Console.WriteLine(x);
        });
        t1.Start();
        x = 20;
    }
}
```

Lambda captures Variable, not the value so the updated value gets printed

Custom object /DTO (Data transfer object)

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(Work);
        t1.Start(
            new UserData
            {
                Id = 1,
                Name = "Samyuktha"
            });
    }
    static void Work(object obj)
    {
        UserData user = (UserData)obj;
        Console.WriteLine($"{user.Id} - {user.Name}");
    }
}

class UserData
{
    public int Id;
    public string Name;
}
```

Modern .NET approach

```
class Program
{
    static void Main(string[] args)
    {
        Task.Run(() =>
        {
            Work("Hello");
        });
    }
    static void Work(object obj)
    {
        Console.WriteLine(obj);
    }
}
```

Thread.Abort - Forcefully terminates thread execution

Thread running — thread.abort() – forceful stop creates multiple problems — unsafe

Problem due to forceful termination:

1.Resource corruption: File left opened, DB transaction half-done, Socket not closed

2.Shared state corruption:

Threads share:Heap, Objects, Variables

Abort mid-operation:partial updates, inconsistent state

3.Lock problems:

Abort inside: lock(obj) { }

Cause: Deadlocks, Unreleased locks

CancellationToken - For safe cancellation of thread

Cooperative cancellation mechanism used to safely request work cancellation

CancellationTokenSource - Creates & controls cancellation

CancellationToken - Passed to worker.

Cancel() - Request cancellation.

IsCancellationRequested - Worker checks signal.

ThrowIfCancellationRequested() - Shortcut API.

```

class Program
{
    static void Main(string[] args)
    {
        CancellationSource cts = new CancellationSource();
        Task worker = Task.Run(() => Work(cts.Token));
        Console.WriteLine(worker.Status); //waiting to run
        Thread.Sleep(5000);
        cts.Cancel();
        worker.Wait();
        Console.WriteLine(worker.Status); //Ran to Completion

        CancellationSource cts2 = new CancellationSource();
        Thread t1 = new Thread(()=>
        {
            Work(cts2.Token);
        });
        Console.WriteLine(t1.ThreadState); //unstarted
        t1.Start();
        Console.WriteLine(t1.ThreadState); //Running
        Thread.Sleep(5000);
        cts2.Cancel();
        Console.ReadLine();
    }

    static void Work(CancellationSource token)
    {
        while (true)
        {
            if (token.IsCancellationRequested)
            {
                Console.WriteLine("Thread Cancelled");
                break;
            }
            Console.WriteLine("Working.....");
            Thread.Sleep(1000);
        }
    }
}

```

ThreadPool

Thread creation is expensive. Why?

- Allocate stack memory
- Create OS thread object
- Register with scheduler
- Manage lifecycle
- Context switching cost

ThreadPool - Is a CLR-managed pool of reusable worker threads used to execute short background work efficiently.

API used

Without parameter - `ThreadPool.QueueUserWorkItem(Work);`

With parameter - `ThreadPool.QueueUserWorkItem(Work,"Hello");`

To check ThreadPool:

```
class Program
{
    static void Main()
    {
        for(int i = 1; i < 10; i++)
        {
            ThreadPool.QueueUserWorkItem(Work,i);
        }
        Console.ReadLine();
    }

    static void Work(object i)
    {
        Console.WriteLine($"Running {i} on thread - {Thread.CurrentThread.ManagedThreadId}");
        Thread.Sleep(1000);
        Console.WriteLine($"Closed {i} on thread - {Thread.CurrentThread.ManagedThreadId}");
    }
}
```

```
Running 2 on thread - 4
Running 3 on thread - 5
Running 4 on thread - 3
Running 1 on thread - 6
Running 5 on thread - 7
Closed 3 on thread - 5
Closed 2 on thread - 4
Running 7 on thread - 4
Running 6 on thread - 5
Closed 4 on thread - 3
Running 8 on thread - 3
Closed 1 on thread - 6
Running 9 on thread - 6
Closed 5 on thread - 7
Closed 7 on thread - 4
Closed 6 on thread - 5
Closed 8 on thread - 3
Closed 9 on thread - 6
```

Creation of Thread involves:

- OS thread creation
- Stack memory allocation
- Scheduler bookkeeping
- Context switching overhead
- Lifecycle management

For performing small task no new Thread() to be created as it's worst case, instead we use **ThreadPool, async await, Task**.

These are preferred because:

- ThreadPool - Reuses the thread (tiny works)
- Async await - Waits until one thread completes and assigns the task (http request, DB handling)
- Task - Single task keeps on running in Background